

MiTienda Pro

Guía Completa del Desarrollador

[Sitio Web](#) · [PWA](#) · [GitHub](#) · [Offline](#) · [APK](#)

Versión 1.3.0 · Edrian Cruz Down

1. Resumen del Sistema

MiTienda Pro es un sistema de ventas online completo con panel de administración, vendedores, tiendas, planes, contactos, links y testimonios. Está construido como una **Progressive Web App (PWA)** que funciona offline en cualquier dispositivo y usa **GitHub como base de datos** (lectura desde *raw.githubusercontent.com*, escritura vía *GitHub REST API* con un Personal Access Token cifrado AES-GCM en el navegador).

Lenguajes y tecnologías

- **HTML5** — estructura semántica del index.
- **CSS3** — diseño dark con variables CSS, gradientes fuego, responsive móvil.
- **JavaScript ES6+ (vanilla)** — lógica de negocio, no usa frameworks.
- **JavaScript ES Modules** — capa PWA/GitHub modular en */lib/*.
- **Service Worker API** — cacheo offline (Cache Storage).
- **IndexedDB** — cola offline, logs, backups, cache de configuración.
- **Web Crypto API (AES-GCM)** — cifrado del token GitHub.
- **GitHub REST API v2022-11-28** — commits, blobs, trees, refs.
- **Canvas API** — fondo animado con partículas.

Estructura del proyecto

Ruta	Descripción
public/app/index.html	HTML semántico (1293 líneas, sin lógica)
public/app/css/styles.css	Todos los estilos del sistema
public/app/js/guard.js	Anti-DevTools, console branding
public/app/js/main.js	Lógica principal: estado S, render, admin
public/app/lib/bootstrap.js	Punto de arranque de capa PWA
public/app/lib/gh-sync.js	Pull/Push de estado contra GitHub
public/app/lib/sync.js	Cola offline → commits agrupados
public/app/lib/github-api.js	Cliente REST de GitHub
public/app/lib/crypto.js	AES-GCM (encriptar token PAT)
public/app/lib/indexeddb.js	Wrapper de IndexedDB
public/app/lib/writer.js	Escritura JSON/imágenes con cola offline
public/app/lib/reader.js	Lectura desde repo público
public/app/lib/image-opt.js	Compresión/redimensión de imágenes
public/app/lib/backup.js	Backups del estado al repo
public/app/lib/repo-init.js	Crea estructura /app /public /private
public/app/lib/admin-config.js	Tarjeta admin GitHub + limpiar caché

public/app/lib/admin-fix.js	Hook para rellenar pestañas admin
public/app/lib/mobile-menu.js	Menú móvil responsive
public/app/service-worker.js	Cache offline (HTML, JS, CSS, fuentes, JSON)
public/app/manifest.json	Manifest PWA (instalable)

2. Configurar la sincronización con GitHub

Esta es la parte **más importante**: sin token PAT, los cambios del admin no se propagan a otros dispositivos.

Paso 1 · Crear el repositorio

- Entra a github.com y crea un repositorio nuevo (público o privado).
- Sube todo el contenido de la carpeta `public/app/` a la raíz del repo.
- Activa **GitHub Pages**: Settings → Pages → Source: main / root.
- Anota la URL pública: `https://<tu-usuario>.github.io/<tu-repo>/`.

Paso 2 · Crear el Personal Access Token (PAT)

- github.com → click tu avatar → **Settings**.
- Menú lateral → **Developer settings**.
- **Personal access tokens** → **Tokens (classic)** → **Generate new token (classic)**.
- Note: MiTiendaPro. Expiration: 90 días o más.
- Scopes: marcar **SOLO repo** (Full control of private repositories).
- Generate token → **copia el ghp_XXXXX... ahora** (no se vuelve a mostrar).

Paso 3 · Configurar en el admin

- Abre tu sitio: `/app/index.html`.
- Botón **Admin** en el header → introduce tu contraseña (default: admin2025).
- Click en pestaña **■ Avanzado**.
- Baja hasta la tarjeta **■ GitHub / Sincronización PWA**.
- Rellena Owner (tu usuario), Repo, Rama (main), pega el Token PAT y elige una Passphrase fuerte.
- Click **■ Guardar config + token** (el token se cifra con AES-GCM con tu passphrase).
- Click **■ Probar conexión** → debe responder **✓ Conectado**.
- Click **■ Crear /app /public /private** (solo la primera vez).

Paso 4 · Verificar sync entre dispositivos

- Desde Windows: cambia el avatar o un plan. Espera 10 segundos.
- Comprueba en GitHub: debe aparecer un nuevo commit en `public/data/state.json`.
- Abre el sitio en otro dispositivo (móvil/tablet) → los cambios aparecen automáticamente.
- Pull periódico: cada 60 segundos y al volver online.

3. Modo Offline en cualquier dispositivo

El Service Worker (service-worker.js v1.3.0) precachea el HTML, CSS, todos los JS, las fuentes Google y los JSON públicos. Tras la primera visita online, el sistema funciona sin internet en **Windows, Android, iPhone, Linux y macOS**.

Estrategias de cache

Recurso	Estrategia	Cache
HTML / navegación	Network-first → fallback a index.html cacheado	mtp-html
CSS / JS app	Stale-while-revalidate	mtp-static
Fuentes Google	Cache-first (largo plazo)	mtp-fonts
api.github.com	Network-first (5s timeout)	mtp-runtime
raw.githubusercontent.com	Stale-while-revalidate (JSON públicos)	mtp-runtime
Imágenes	Cache-first	mtp-img

Probar offline

- **Chrome/Edge:** F12 → Application → Service Workers → marca *Offline* → recarga.
- **Android:** visita el sitio, activa modo avión, recarga la pestaña.
- **iPhone:** visita el sitio en Safari, activa modo avión, recarga.
- Si ves pantalla blanca: borra caché (Admin → Avanzado → ■ Limpiar caché PWA) y vuelve a entrar online una vez.

Limpiar la caché

- Admin → Avanzado → tarjeta GitHub → botón ■ **Limpiar caché PWA**.
- Desregistra el Service Worker, borra todos los Cache Storage y recarga.
- Útil tras actualizar el código del sitio para que los usuarios reciban la última versión.

4. Cómo desplegar el servidor

Opción A · GitHub Pages (gratis, recomendado)

- Repositorio público → Settings → Pages → Source: main / / (root).
- Espera 1-2 minutos. URL: <https://<usuario>.github.io/<repo>/app/index.html>.
- Custom domain opcional: añade CNAME con tu dominio.
- HTTPS automático (requisito para PWA + Service Worker).

Opción B · Netlify / Vercel / Cloudflare Pages

- Conecta el repositorio. Build command: vacío. Publish directory: public/app.
- Despliegue automático en cada push a main.

Opción C · Servidor propio (Nginx / Apache)

Sirve la carpeta public/app/ como sitio estático. Configuración mínima Nginx:

```
server {
    listen 443 ssl http2;
    server_name mitiendapro.com;
    ssl_certificate      /etc/letsencrypt/live/mitiendapro.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/mitiendapro.com/privkey.pem;
    root /var/www/mitiendapro/app;
    index index.html;
    location / { try_files $uri $uri/ /index.html; }
    location /service-worker.js {
        add_header Cache-Control "no-cache, no-store, must-revalidate";
    }
}
```

Opción D · Local con Python (desarrollo)

```
cd public/app && python3 -m http.server 8080
# Abre http://localhost:8080
```

5. Generar la APK Android (TWA)

Convertimos la PWA en una **Trusted Web Activity** (TWA) usando Bubblewrap. El resultado es un .apk/.aab instalable o publicable en Google Play.

Requisitos

- Node.js 18+ y Java JDK 17.
- Sitio publicado con HTTPS (GitHub Pages vale).
- Manifest PWA válido (manifest.json) — ya incluido.

Pasos con Bubblewrap

```
# 1. Instalar Bubblewrap
npm install -g @bubblewrap/cli

# 2. Inicializar el proyecto TWA (apunta a tu manifest publicado)
bubblewrap init --manifest=https://<usuario>.github.io/<repo>/app/manifest.json

# 3. Construir la APK
bubblewrap build

# 4. APK firmada generada en ./app-release-signed.apk
# 5. Para publicar en Play Store, usar el .aab:
bubblewrap build --release
```

Alternativa rápida · PWABuilder.com

- Entra en pwabuilder.com.
- Pega la URL de tu sitio (con HTTPS).
- Click **Package For Stores** → **Android**.
- Descarga el zip con la APK firmada y las instrucciones de Digital Asset Links.

Vinculación de dominio (Digital Asset Links)

Para que la APK abra sin barra de URL, sube el archivo .well-known/assetlinks.json generado por Bubblewrap a la raíz de tu sitio.

6. Ruta de estudio para programadores

Plan ordenado de lo más simple a lo más complejo. Cada nivel se apoya en el anterior.

Nivel 1 · Landing Page / Portafolio

- **HTML5 + CSS3**: semántica, Flexbox, Grid, media queries.
- **JavaScript vanilla**: DOM, eventos, fetch.
- **Frameworks CSS**: Tailwind CSS o Bootstrap.
- **Hosting**: GitHub Pages, Netlify, Vercel.
- Proyectos: portafolio personal, landing de producto, blog estático.
- Recursos: *freeCodeCamp*, *MDN Web Docs*, *Frontend Mentor*.

Nivel 2 · iComerce (e-commerce)

- **React o Vue 3 + Vite**.
- Estado: **Zustand**, **Pinia** o **Redux Toolkit**.
- Routing: **React Router / Vue Router**.
- Backend: **Node.js + Express**, **Strapi** o **Supabase**.
- DB: **PostgreSQL** o **MongoDB**.
- Pagos: **Stripe**, **PayPal**, integraciones locales (Transfermóvil en Cuba).
- Carrito persistente, inventario, checkout, panel admin.
- Recursos: curso oficial de React, *fullstackopen.com*, *Platzi*.

Nivel 3 · Chat Online (tiempo real)

- **WebSockets**: Socket.IO o WS nativo.
- Backend: Node.js + Socket.IO + Redis (pub/sub para escalar).
- Auth: JWT + refresh tokens, OAuth (Google).
- DB: mensajes en MongoDB / PostgreSQL con índice por sala.
- Frontend: React + hooks, scroll infinito, reconexión automática.
- Notificaciones push: Web Push API + Firebase Cloud Messaging.
- Cifrado E2E opcional: Web Crypto API (igual que MiTienda Pro usa para el token).

Nivel 4 · Red social tipo Facebook

- Arquitectura: monorepo con **Turborepo**, microservicios.
- Backend: **NestJS** + **GraphQL** (Apollo) + **Prisma ORM**.
- Feed: algoritmo de ranking, paginación con cursores.
- Storage: **S3 / Cloudflare R2** para imágenes y vídeos.
- Caché: **Redis** (sesiones, feed, contadores).

- Búsqueda: **Elasticsearch** o **Meilisearch**.
- Mobile: **React Native** o **Flutter**.
- DevOps: Docker, Kubernetes, CI/CD con GitHub Actions.

7. Ideas de sistemas con impacto real

Proyectos viables y con demanda concreta. Ordenados por complejidad y por país.

■ ■ Cuba — alta prioridad por escasez y conectividad limitada

- **Catálogo offline de MLC/USD:** precios actualizados de tiendas, sincronización cuando hay datos.
- **Mensajería P2P sin internet:** Bluetooth/WiFi Direct (Bridgefy-like) para zonas sin red.
- **Mapa colaborativo de colas:** usuarios reportan disponibilidad de productos y combustible.
- **Pasarela local Transfermóvil/EnZona:** integrar pagos para e-commerce cubanos.
- **Clases offline-first:** contenido educativo cacheado tipo Khan Academy local.
- **Marketplace de servicios profesionales:** cuentapropistas (plomeros, electricistas, repostería).
- **Agregador de transporte privado:** rutas fijas de boteros y máquinas.

■ ■ Argentina — inflación e informalidad

- **Lista de precios histórica:** comparador supermercados con gráficos de inflación.
- **Gestor de monotributistas:** facturación AFIP, vencimientos, recordatorios.
- **Cripto/dólar tracker:** cotizaciones blue/MEP/CCL con alertas push.
- **Marketplace de trueque local:** para barrios y ferias americanas.
- **Plataforma de cooperativas:** gestión interna, asambleas, votos.

■ ■ España — autónomos y servicios

- **Facturación para autónomos:** alta IRPF, modelo 130/303, recurrentes.
- **Reservas para pequeños negocios:** peluquerías, bares, talleres (sin comisión por reserva).
- **Marketplace de proximidad:** productos km0, agricultura local, pedidos al pueblo.
- **Agregador de ayudas y subvenciones:** filtra por CCAA, sector, antigüedad.
- **Plataforma de aprendizaje en oficios:** vídeo + comunidad + tutorías.

Universal · útil en cualquier país

- Sistema de gestión escolar offline-first.
- Historia clínica digital portable (paciente dueño de sus datos).
- Plataforma de microtarefas remuneradas en cripto.
- Red de denuncias ciudadanas con verificación distribuida.

8. Anexo técnico

Cifrado del token (AES-GCM)

El PAT nunca se guarda en claro. Se cifra con tu passphrase usando PBKDF2 (100k iteraciones) + AES-GCM 256 bits via Web Crypto API. Sin la passphrase, el token es irre recuperable incluso con acceso al IndexedDB.

Estado sincronizado (state.json)

El admin guarda en public/data/state.json los campos: config, plans, ownerWeb, ownerApk, contacts, links, avatar, clientStores, terms, promoConfig, testimonials, sitelcon, shopUrl, jsonAutoUrl, contractTemplate, freeDesc, freeFeats, más un timestamp __ts. Se excluyen adminPass, recoverAnswer y imágenes base64 mayores a 200 KB.

Resolución de conflictos

Last-write-wins por timestamp. Si el remoto tiene __ts mayor que el local, gana el remoto. Si son iguales o local es mayor, gana el local. Para evitar pérdidas: editar en un solo dispositivo a la vez.

Comandos útiles del navegador

```
// Verificar Service Worker
navigator.serviceWorker.getRegistrations().then(console.log);

// Limpiar todos los caches
caches.keys().then(ks => ks.forEach(k => caches.delete(k)));

// Forzar push del estado a GitHub
window.MTP.GhSync.pushNow();

// Forzar pull desde GitHub
window.MTP.GhSync.pull().then(console.log);

// Ver cola offline pendiente
window.MTP.DB.listQueue().then(console.log);

// Ver logs internos
window.MTP.DB.listLogs(50).then(console.table);
```

Solución de problemas

- **Pantalla blanca offline:** ya está corregida en SW v1.3.0 (fallback a index.html).
- **Cambios no aparecen en otro dispositivo:** verifica que el commit llegó al repo. Comprueba que el otro dispositivo tiene Owner+Repo configurados.
- **'Token inválido':** el PAT expiró. Genera uno nuevo en GitHub y guárdalo de nuevo.
- **'Encriptación falló':** passphrase incorrecta. Si la olvidaste, usa *Olvidar token* y vuelve a guardarlo.
- **SW no se actualiza:** usa el botón ■ Limpiar caché PWA del admin.

Fin de la documentación — MiTienda Pro v1.3.0

© Edrian Cruz Down — Todos los derechos reservados